

Q.No:2

DS	Session	Tree 1	Question Information	Level 1	Challenge 62
Question description					
There are three common ways to traverse the nodes of a binary tree:					
• <i>Preorder</i>: First process the root, then the left subtree, and finally the right subtree. • <i>Inorder</i>: First process the left subtree, then the root, and finally the right subtree. • <i>Postorder</i>: First process the left subtree, then the right subtree, and finally the root.					
There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal.					
Constraints					
• $1 \leq n \leq 10^5$					
Input					
The first input line has an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.					
After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers.					
You can assume that the input corresponds to a binary tree.					
Output					
Print the postorder traversal of the tree.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 5 3 2 1 4 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5	INPUT (STDIN) 6 6 5 3 2 1 4 6 3 5 1 2 4 EXPECTED OUTPUT 3 1 4 2 5 6

Mandatory Test Cases

Test Case 1
KEYWORD for(i=1; i<=n; i++)

Code:

```
#include <stdio.h>

int pre[100000];
int in[100000];
int pos[100001];

void postorder(int preStart, int inStart, int inEnd)
{
    if (inStart > inEnd)
        return;
```

```
int root = pre[preStart];

int mid = pos[root];

int leftSize = mid - inStart;
postorder(preStart + 1, inStart, mid - 1);

postorder(preStart + leftSize + 1, mid + 1, inEnd);
printf("%d ", root);
}

int main()
{
    int n;

    scanf("%d", &n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &pre[i]);
    }
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &in[i]);
        pos[in[i]] = i;
    }

    postorder(0, 0, n - 1);

    return 0;
}
```

Output:

Test_case 1:

```
5
5 3 2 1 4
3 5 1 2 4
3 1 4 2 5 %
```

Test_case 2:

```
6
6 5 3 2 1 4
6 3 5 1 2 4
3 1 4 2 5 6 %
```